

Locking the Shared Arm

Day 4: Network and OS Foundations for Safe
Robotic Actuation

Mentor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu

July 2026

Summer Course at Lanzhou University
The Hong Kong Polytechnic University (PolyU)

Table of content

1. Threads and Shared State
2. Mutex: the Simplest Lock
3. Semaphores and Reader/Writer Locks
4. Deadlock
5. Priority Inversion
6. Project Hands-on Preview: Lock the Bridge

Three Days In, One Arm Out

The view so far.

A frame format with CRC (Day 1), a pub-sub bridge with QoS (Day 2), a `poll()` server fielding many clients (Day 3). The bridge sees many talkers.

The arm does not.

Downstream of every multiplexer sits one serial fd, driving one physical actuator. The number of joints does not multiply because the number of plans did.

Today's question.

When two threads inside your bridge both want to drive the arm, what does the OS do, and how do we make sure no two plans move the same joint at once?



Threads and Shared State

Why Threads at All

Yesterday's lesson stands.

Multiplexing with `poll()` handled many *clients* without threads. One thread can wait on many fds; that is correct, and Day 5 doubles down on it.

Threads are for work, not waiting.

Spawn a thread when you have CPU-bound or blocking work that should run *in parallel* with the event loop. A CRC over a megabyte payload, a sandbox check that takes 50 ms, a logger that calls `fsync` on every entry.

Today's threads in particular.

The lab gives you a planner thread that produces motion frames and a safety thread that injects e-stop frames. Both write to the same serial fd.

What Threads Share, What They Do Not

Shared by default.

Heap allocations, global variables, file descriptors, signal handlers. The whole address space is one. Yesterday's `serial_fd` sitting in a global is reachable from every thread.

Per-thread.

Stack, CPU registers, `errno` (on POSIX). Local variables are private; pointers to them are not.

The freedom and the trap.

Sharing is free, fast, and the exact reason threads are dangerous. With multiple processes (yesterday's `prefork`) you had to work to share state; here you have to work to *not*.

The Race in 12 Lines

```
1 /* shared: int serial_fd; */
2 void *planner(void *arg) {           /* thread A */
3     while (1) {
4         frame_t f = build_move(...);
5         write(serial_fd, &f, sizeof f);
6     }
7 }
8 void *safety(void *arg) {           /* thread B */
9     while (1) {
10        frame_t e = build_estop();
11        write(serial_fd, &e, sizeof e);
12    }
13 }
```

The Race in 12 Lines (Cont'd)

The good news.

`write()` on a pipe is atomic for small payloads on Linux.

The bad news, almost everywhere else.

A serial port is not a pipe. Two interleaved `write()`s can yield half of frame A followed by half of frame B on the wire.

The visible symptom.

Day 1's CRC catches it, the simulator NACKs, the arm does not move on either command. Safe by accident; we want safe by design.

What a Race Condition Is

The textbook definition.

Two or more threads access shared state, at least one writes, and the final result depends on the order in which the OS scheduler runs them. Run the same program twice, get two different answers.

The classic example.

Two threads execute `counter++`. That is read, increment, write, three machine instructions. Interleaved naively, the counter advances by one instead of two.

Our example.

Two threads execute "frame the command, write the bytes". Interleaved naively, the wire carries garbage. Same shape, higher stakes.

Critical Section

The named region.

A stretch of code that accesses shared state and must not be interleaved with another thread's stretch over the same state. Outside the critical section, threads run freely; inside, exactly one at a time.

Three properties any solution needs.

Mutual exclusion (at most one thread inside), *progress* (someone gets in if anyone is waiting), *bounded waiting* (no thread waits forever).

Where this lecture lives.

Mutex, semaphore, reader-writer lock are mechanisms for enforcing those three properties. The hard part is choosing the right one.



Mutex: the Simplest Lock

pthread_mutex_t

The smallest lock that works.

One bit of state (locked or not), plus a queue of threads waiting their turn.

```
1 pthread_mutex_t serial_lock = PTHREAD_MUTEX_INITIALIZER;
2
3 void send_frame(const frame_t *f) {
4     pthread_mutex_lock(&serial_lock);
5     write(serial_fd, f, sizeof *f);
6     pthread_mutex_unlock(&serial_lock);
7 }
```

Reading the loop.

Both planner and safety threads now route every write through `send_frame`. Inside the lock, exactly one thread at a time touches the serial port. No more half-frames.

What the Kernel Does for You

`pthread_mutex_lock` is two paths.

Fast path: the lock is free, an atomic compare-and-swap takes it, no syscall. Slow path: the lock is held, the thread blocks via the kernel's `futex` mechanism (Day 3's wait queue, again).

`pthread_mutex_unlock` is the mirror.

If anyone is waiting, the unlock call wakes *one* of them; the others stay parked.

The reading.

Mutex contention is cheap when rare and expensive when frequent. If two threads fight for the serial port a thousand times per second, you are paying syscalls; the cure is not a faster lock but a queue, or fewer threads.

Choosing the Granularity

One lock per resource, not one lock total.

Different shared objects deserve different locks. The serial fd has one lock; the in-memory log has another; the topic subscription table has a third.

The two failure modes.

Too coarse: every thread serialises behind one global lock, defeating parallelism. Too fine: many locks, more code to think about, more chances for the deadlock you will meet shortly.

The robotics reading.

Our arm has *one* serial fd downstream, so one lock around the wire is right. If we had three actuators on three ports, three independent locks would let three motions proceed in parallel, exactly what the HCR handbook calls *single-actuator safety*.



Semaphores and Reader/Writer Locks

Mutex vs Semaphore

Mutex is binary, and it has an owner.

The thread that locked must be the thread that unlocks. Think of it as a key in a pocket: the holder is named.

Semaphore counts.

An integer that threads decrement (`sem_wait`) and increment (`sem_post`). When it reaches zero, future waiters block until somebody posts.

What the count expresses.

Resources of which there are N identical copies. Five worker slots. Three buffer pages. Zero (initially), meaning "wait until someone signals you". Same primitive, different metaphor.

When Each Is Right

Use a mutex for exclusive access to one resource.

The serial fd. A shared data structure. A pointer that nobody else may dereference while you mutate it.

Use a counting semaphore for bounded pools.

Three pre-allocated frame buffers; `sem_wait` blocks the planner if all three are in flight. The bounded-buffer queue from yesterday's backpressure discussion, made explicit.

Use a binary semaphore for one-shot signalling.

Initialise to 0; one thread `sem_wait`s for an event; another thread `sem_post`s when ready. A mutex cannot express this cleanly because there is no shared resource being guarded.

Producer-Consumer with Two Semaphores

The pattern.

Planner thread produces motion frames into a bounded queue; bridge thread consumes them and writes the wire. The queue holds N slots; producer must not overflow it, consumer must not read empty.

Two counting semaphores plus a mutex.

empty starts at N , full starts at 0. Producer: `sem_wait(empty)`, lock, push, unlock, `sem_post(full)`. Consumer: `sem_wait(full)`, lock, pop, unlock, `sem_post(empty)`.

Why this matters Friday.

The gatekeeper is exactly this shape: many publishers, one wire, one actuator speed. Semaphores are how backpressure (Day 3) meets fairness.

Reader-Writer Locks

The asymmetry many readers want.

Reading is non-destructive; many threads can read the same data without conflict. A reader-writer lock allows *any number* of concurrent readers *or* exactly one writer.

The robot bus example.

arm/state is published once per cycle by the bridge; subscribed by a dashboard, a logger, a planner, a safety monitor. With a mutex they serialise; with a reader-writer lock they read in parallel and only block when the bridge writes.

The trade-off.

Readers starve if writers always preempt; writers starve if readers keep arriving. `pthread_rwlock_t` picks one policy; choose by which side you cannot afford to delay.



Deadlock

When Locks Themselves Become the Problem

The story.

Thread A holds lock 1, wants lock 2. Thread B holds lock 2, wants lock 1. Neither can make progress. They will wait forever, costing no CPU, looking healthy in top.

What it looks like in your bridge.

Planner thread grabs the serial lock, then needs the log lock. Logger thread grabs the log lock, then needs the serial lock for an audit entry. Done.

Why this is worse than a crash.

A crash sets off alarms. A deadlock is silent: queues stop draining, latency goes to infinity, but no error appears. Day 5's e-stop deadline measurement will catch it; nothing else automatically will.

The Four Coffman Conditions

Deadlock needs all four, simultaneously.

1. *Mutual exclusion*. Some resource cannot be shared.
2. *Hold and wait*. A thread holds one resource and waits for another.
3. *No preemption*. The OS will not take a lock back from a thread.
4. *Circular wait*. Threads form a cycle, each waiting on the next.

Why the list is useful.

Break any one condition and deadlock cannot occur. Most prevention strategies attack *circular wait*, because the other three are usually requirements of the program.

Five Philosophers, Five Forks

The story you have probably heard.

Five philosophers sit around a table. Five forks between them, one between each neighbour. To eat, a philosopher needs the fork on the left *and* the fork on the right.

The deadlock.

Each philosopher picks up their left fork, then reaches for the right. Every right fork is now someone else's left fork. Nobody eats. All four Coffman conditions, in one image.

The cure that will appear again in three slides.

Number the forks 1 to 5. Any philosopher reaching for two forks takes the lower-numbered one first. Circular wait broken; everyone eats; nobody starves. The same prescription for your bridge.

Constructing a Deadlock

```
1 pthread_mutex_t A, B;
2 void *t1(void *arg) {
3     pthread_mutex_lock(&A);
4     usleep(1000);           /* let t2 race */
5     pthread_mutex_lock(&B); /* (*) waits */
6     pthread_mutex_unlock(&B);
7     pthread_mutex_unlock(&A);
8 }
9 void *t2(void *arg) {
10    pthread_mutex_lock(&B);
11    usleep(1000);
12    pthread_mutex_lock(&A); /* (*) waits */
13    pthread_mutex_unlock(&A);
14    pthread_mutex_unlock(&B);
15 }
```


Constructing a Deadlock (Cont'd)

Why this hangs.

Both threads enter their `usleep` holding one lock. They each wake and ask for the other thread's lock. Both `pthread_mutex_lock` calls now block forever, on each other.

The hands-on hour will make you write this.

Confirming the four Coffman conditions on your own code is the only reliable way to recognise them later. A deadlock you did not produce yourself looks like "the program is slow".

The simplest cure is also the best.

Lock ordering. If every thread, everywhere, takes A before B, there is no cycle, and condition 4 is broken.

Prevention by Lock Ordering

Assign each lock a number.

`serial_lock = 1`, `log_lock = 2`, `subs_table = 3`. Any code that takes more than one lock takes them in ascending order. Never descend.

What this costs.

Discipline, and one code review pass per call site. You may have to release and re-acquire to satisfy the order; that is fine, the alternative is deadlock.

What this guarantees.

No cycle is possible, so no deadlock. The other three Coffman conditions remain, and that is correct; this strategy targets condition 4 surgically and leaves the rest of the program alone.

Detection vs Prevention

Prevention.

Design the locks so deadlock cannot arise. Lock ordering, lock-free data structures, or simply not using more than one lock. Today's preferred answer.

Avoidance.

At each lock request, the OS checks whether granting it could lead to a future deadlock. The classical algorithm (Banker's) requires knowing every thread's maximum needs in advance, which we usually do not.

Detection and recovery.

Let deadlock happen, scan periodically for cycles in the wait graph, kill a thread to break it. Databases do this; safety-critical robot code emphatically does not.

In-Class Quiz: Which Cure Removes the Deadlock?

Threads A and B currently take mutexes in opposite orders, causing a hang. Which change *guarantees* no deadlock with no cost to correctness?

1. Add a `usleep(100)` between the two lock calls.
2. Make both mutexes *recursive* (a thread may relock its own mutex).
3. Require both threads to lock the two mutexes in the same fixed order.
4. Use `pthread_mutex_trylock` and abandon if either call returns busy.

In-Class Quiz: Answer

Answer. 3

Why.

Option 3 breaks Coffman condition 4 (no cycle is possible). Option 1 hides the bug under timing. Option 2 helps recursive call patterns but does nothing for the cross-thread cycle. Option 4 avoids the hang but turns the deadlock into livelock and lost work; correctness suffers.

The reading.

Of the four conditions, *circular wait* is the cheapest to attack, and the one with no downside.



Priority Inversion

The Real-Time Wrinkle

The setup.

Three threads, three priorities: high (safety), medium (housekeeping), low (logger). A mutex protects the audit log. The logger thread holds it; the safety thread needs it.

What goes wrong.

The medium thread, which does not touch the lock at all, preempts the logger and runs. The logger cannot release the lock. The safety thread, the highest priority in the system, waits behind a thread of lower priority.

The name.

Priority inversion: the system's effective priority order is upside down for as long as the lock is held.

The Mars Pathfinder Story

Engineering reality, not folklore.

In 1997 the Mars Pathfinder rover repeatedly reset itself. Cause: a high-priority bus management task waited for a mutex held by a low-priority weather task, while a medium-priority communications task ran indefinitely. Classic priority inversion.

The fix.

NASA enabled *priority inheritance* on the lock: while a low-priority thread holds it, the OS raises that thread to the priority of the highest waiter, until it releases.

The lesson for our arm.

Priority inversion is a property of the lock *plus* the scheduler. Choosing a primitive without thinking about both is the bet NASA lost.

Priority Inheritance and Ceiling

Priority inheritance.

Whoever holds the lock runs at the maximum priority of any thread waiting on it. Reverts on unlock. Implemented by Linux with `PTHREAD_PRIO_INHERIT`.

Priority ceiling.

Each lock is assigned a ceiling priority equal to the highest priority of any thread that will ever request it. Holding the lock raises the holder to that ceiling. Implemented with `PTHREAD_PRIO_PROTECT`.

When to ask for either.

Soft real-time systems usually do not need them. The moment a higher-priority thread *that does not touch your locks* can preempt the holder, you are on the inversion road.

Safety-Aware Scheduling at the Platform

From the HCR handbook (Chapter 5).

Platform-level "priority" is not one number: the handbook uses a 5-factor formula combining safety urgency, deadline, operator authority, plan freshness and fairness.

Two principles even with one arm.

Preemption: a higher-priority plan can interrupt a lower-priority one. *Fairness*: no plan starves behind a stream of higher-priority work.

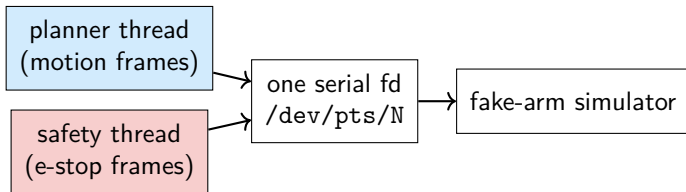
The lock interaction.

Preemption is the mechanism that makes inversion possible: a preempted holder cannot release. Friday's gatekeeper needs inheritance on the wire lock the moment the planner outranks the logger.



Project Hands-on Preview: Lock the Bridge

The Hands-on Chain, Today



Two threads, one wire.

Both threads produce well-formed Day 1 frames. Without a lock, the wire mixes their bytes; the simulator sees CRC failures and NACKs.

The HCR parallel.

This is the smallest possible model of *single-actuator safety*: many plans may exist, exactly one reaches the joint at a time.

Hands-on Tasks

Task 1: produce the corruption.

Run planner + safety threads with no lock around `write(serial_fd, ...)`. Count NACKs from the simulator over 30 seconds; this is your baseline.

Task 2: a mutex around the wire.

Add the `send_frame` pattern from this morning. Repeat the run. NACK count should fall to zero; throughput should not drop visibly.

Task 3: construct a deadlock, then fix it.

Add a second mutex (the audit log). Have the threads take the two locks in opposite orders. Watch the bridge stall, then resolve it by lock ordering. Bring your before/after to the wrap-up.

Hands-on Tasks (Cont'd)

Pass condition.

Zero NACKs under the locked baseline, and a deadlock you can produce and cure on demand.

Keep what you build.

Tomorrow's `epoll` rewrite makes most of today's locks unnecessary by being single-threaded; the capstone gatekeeper inherits exactly one of them, around the wire.

Day 4 Summary

Threads share by default.

Heap, globals and fds are visible to every thread. A serial fd touched by two threads without a lock is a race with safety consequences.

Locks fix races, locks cause deadlocks.

Mutex for one resource, semaphore for a pool, reader-writer for many-reader workloads. Pick a lock order if you take more than one.

Priority inversion is the real-time trap.

A high thread can wait behind a low one through a lock; priority inheritance is the cure.

Tomorrow.

Single-threaded `epoll` sidesteps most of today's problems; we measure an e-stop against a strict deadline.

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template